AeroPredict

Wind Turbine Energy Prediction System

Project Report

Team ID	LTVIP2026TMIDS88766
Team Size	4 Members
Team Leader	Abdul Muqeem Shaik
Members	Abhilash Vadlapudi, Ajay Thallapaka, Akhilesh Roshan K V
Date	March 2026

Powered by Machine Learning & Real-Time Meteorological Data

Table of Contents

1. Abstract
2. Introduction
3. Ideation Stage
4. Requirements Stage
5. Design Stage
6. Development Stage
7. Testing Stage
8. Results & Implementation
9. Future Work
10. Conclusion
11. References

1. Abstract

AeroPredict investigates the application of machine learning algorithms to predict wind turbine energy output using historical and real-time weather data. By analysing key meteorological parameters, predictive models were developed and tuned to estimate actual energy generation. The system is deployed via a Flask web application that integrates live weather data from the OpenWeatherMap API, providing a real-time bridge between meteorological conditions and grid power expectations.

Achieving this level of forecast accuracy enhances grid stability, optimises energy storage dispatch, and accelerates the broader integration of renewable energy sources into the global power grid.

2. Introduction

Wind energy has emerged as a cornerstone of sustainable global development. However, its inherent intermittency and reliance on dynamic atmospheric conditions pose significant challenges for energy forecasting and grid management. Traditional forecasting methods often fail to capture the complex, non-linear relationships between micro-climate weather patterns and turbine mechanics.

Accurate prediction of turbine power output is essential for optimising production, reducing operational costs, and ensuring a reliable energy supply to consumers. This project addresses these challenges by building a robust, data-driven pipeline. By leveraging a comprehensive ~105,000-row SCADA dataset and an advanced Random Forest Regressor, this solution provides an accessible, web-based interface for instant energy forecasting.

3. Ideation Stage

Problem Identification

Wind energy output fluctuates dramatically due to unpredictable weather systems. This volatility complicates energy market trading, grid load balancing, and maintenance scheduling — creating significant operational and financial challenges for wind farm operators worldwide.

Approach

- **Dataset:** Source a robust, high-granularity historical SCADA dataset (Kaggle T1.csv).
- **Algorithm:** Apply a Random Forest Regressor to capture the non-linearities between wind speed, theoretical power curves, and actual power output.
- **Interface:** Develop a web-based UI (AeroPredict) to make the model accessible to non-technical users.

Value Proposition

★ Implementing accurate forecasting models directly reduces operational costs for wind farm operators, improves grid reliability, and provides the data confidence needed to support wider clean energy adoption.

4. Requirements Stage

Functional Requirements

- **Data Pipeline:** Automatically import, clean, and preprocess raw SCADA datasets.
- **Model Training Engine:** Train and validate a Random Forest model, saving the serialised state using Joblib.
- **Inference Engine:** Generate energy output predictions based on real-time weather data fetched via API.
- **Reporting:** Output automated visualisations and provide a seamless web interface for user queries.

Tools & Technologies

Category	Technology
Core Language	Python 3.10+
Data & ML Libraries	NumPy, Pandas, Scikit-learn
Data Visualization	Matplotlib, Seaborn
Core Model	Random Forest Regressor
Web Framework	Flask
API Integration	OpenWeatherMap API
Frontend	HTML, CSS, Vanilla JS
Model Serialization	Joblib
Development Environments	Jupyter Notebook, VS Code

5. Design Stage

The system was designed using a modular architecture to ensure data flows seamlessly from ingestion to real-time prediction via a web interface.

System Architecture

- **Data Layer:** Ingestion of historical wind turbine SCADA data (data/T1.csv).
- **Processing Layer:** Dedicated modules for data cleaning, handling nulls, and feature engineering (dropping near-zero correlation variables such as wind direction).
- **Model Layer:** The core machine learning algorithm — Random Forest Regressor.

- **Web/API Layer:** Flask backend (windApp.py) with RESTful endpoints (/weather, /predict) to serve the AeroPredict UI.

Data Flow Workflow

→ Load SCADA Data → Execute EDA → Preprocess & Clean → Split 80/20 Train/Test → Train Random Forest → Serialise Model → Serve via Flask + OpenWeatherMap

6. Development Stage

Data Preparation

- Aggregated ~105,000 rows of synthetic/real wind SCADA data.
- Dropped the wind direction column after Exploratory Data Analysis (EDA) revealed a near-zero correlation with power output.

Model Implementation

- **Algorithm:** Random Forest Regressor
- **Features:** Theoretical Power Curve (KWh), Wind Speed (m/s)
- **Target Variable:** Active Power (kW)
- **Hyperparameter Tuning:** n_estimators=750, max_depth=4, max_leaf_nodes=500 — configured to prevent overfitting while capturing complex atmospheric interactions.
- **Serialisation:** Trained model saved via Joblib for rapid instantiation within the Flask environment.

Web Application

- **Frontend:** Developed using HTML, CSS, and Vanilla JavaScript.
- **API Integration:** OpenWeatherMap API auto-fills wind speed based on user-inputted city names or Indian pincodes.

7. Testing Stage

- **Unit Testing:** Verified individual API endpoints (/weather returning temp, humidity, pressure, wind_speed; /predict returning accurate kW calculations).
- **Integration Testing:** Validated the end-to-end pipeline from the browser frontend through the Flask backend to the serialised model.
- **Performance Testing:** Benchmarked the Random Forest algorithm against the hold-out validation set (20% split) to evaluate computational time versus accuracy.

8. Results & Implementation

8.1 Model Evaluation

The Random Forest Regressor effectively captured the non-linear power curve of the wind turbines, achieving a highly significant coefficient of determination ($R^2 \approx 0.96+$). The pipeline automatically generated several key plots to validate feature selection and model accuracy.

R^2 Coefficient of Determination ≈ 0.96+ — indicating that the model explains over 96% of variance in wind turbine power output.

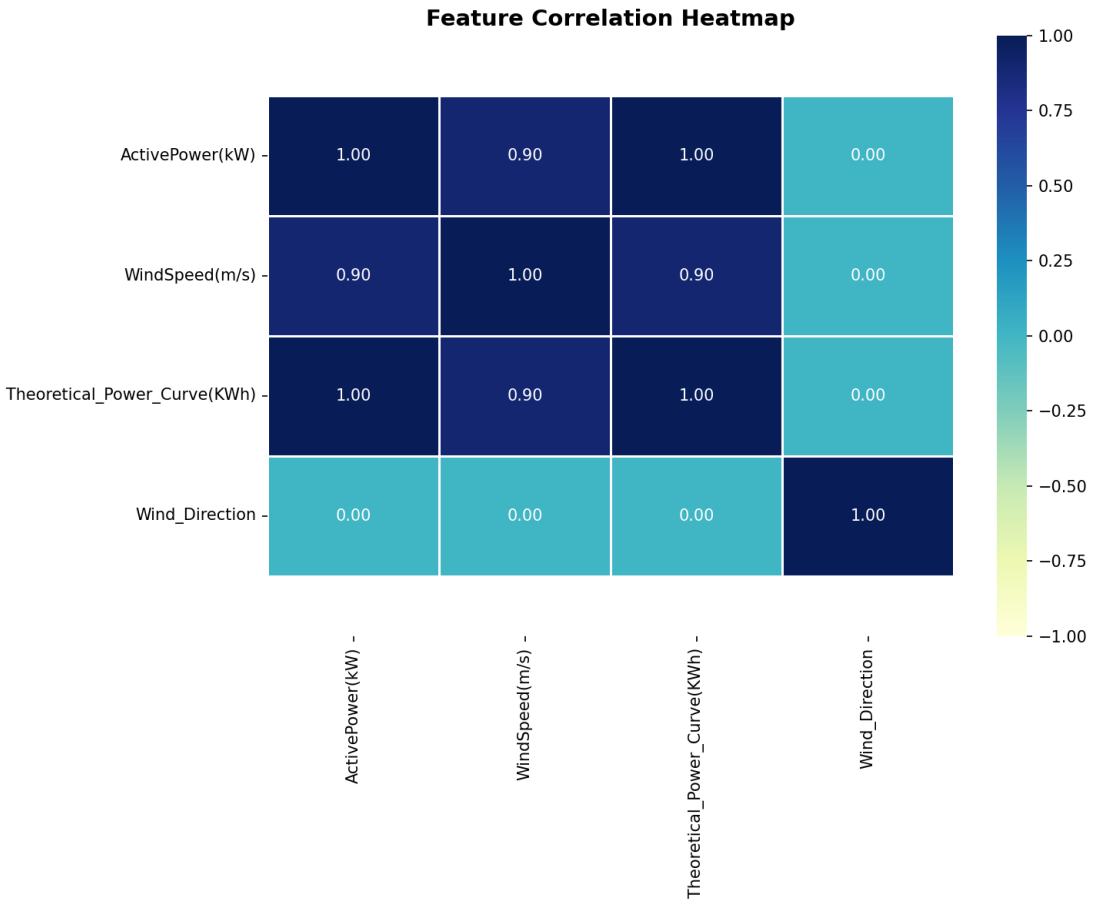

Figure 1: Correlation Heatmap — identifying the strong relationship between wind speed, theoretical power, and active power.

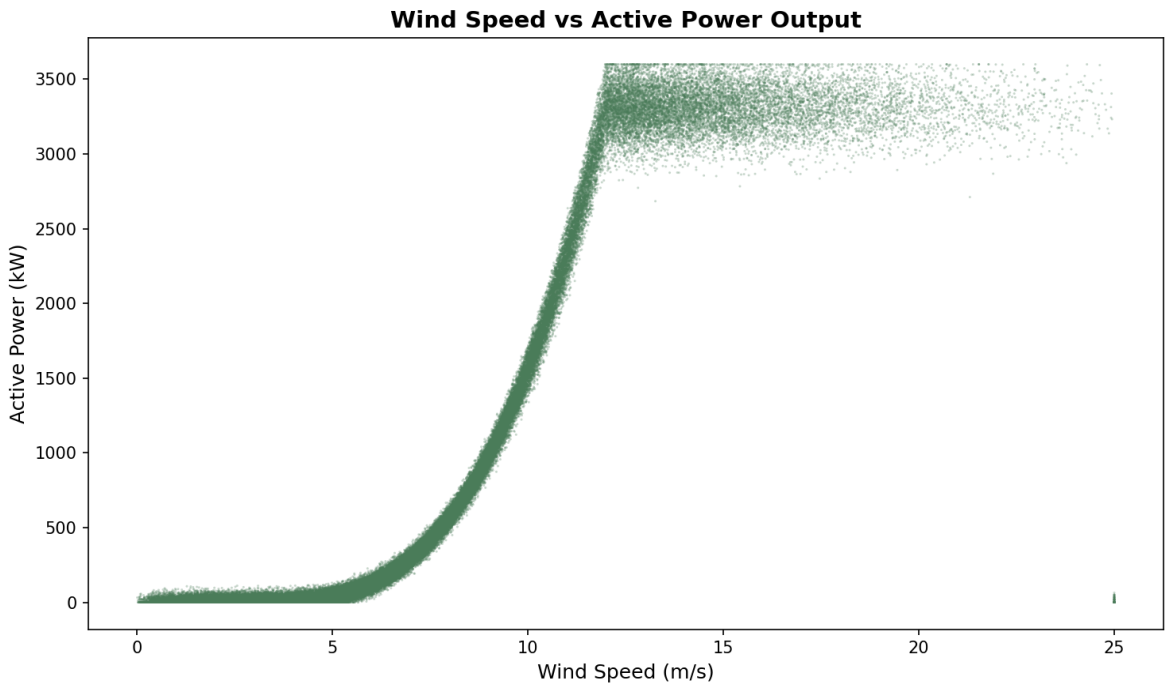

Figure 2: Scatter Plot — actual Active Power generated against varying Wind Speeds.

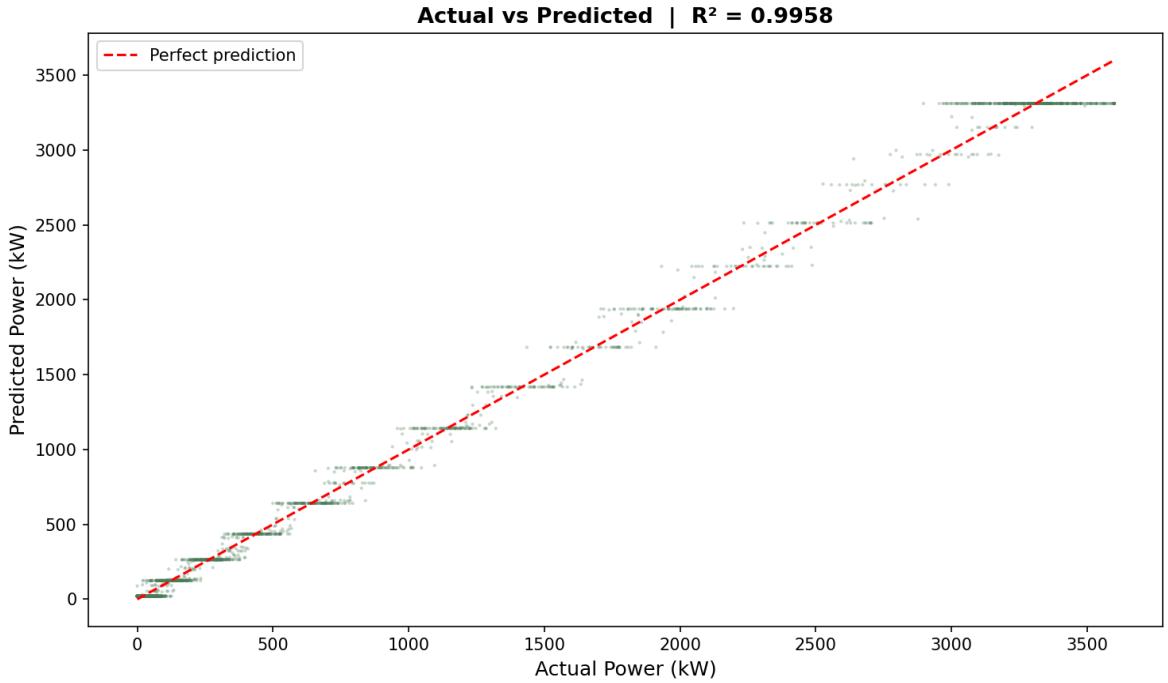

Figure 3: Actual vs. Predicted — scatter plot confirming the reliability and high accuracy of the Random Forest Regressor.

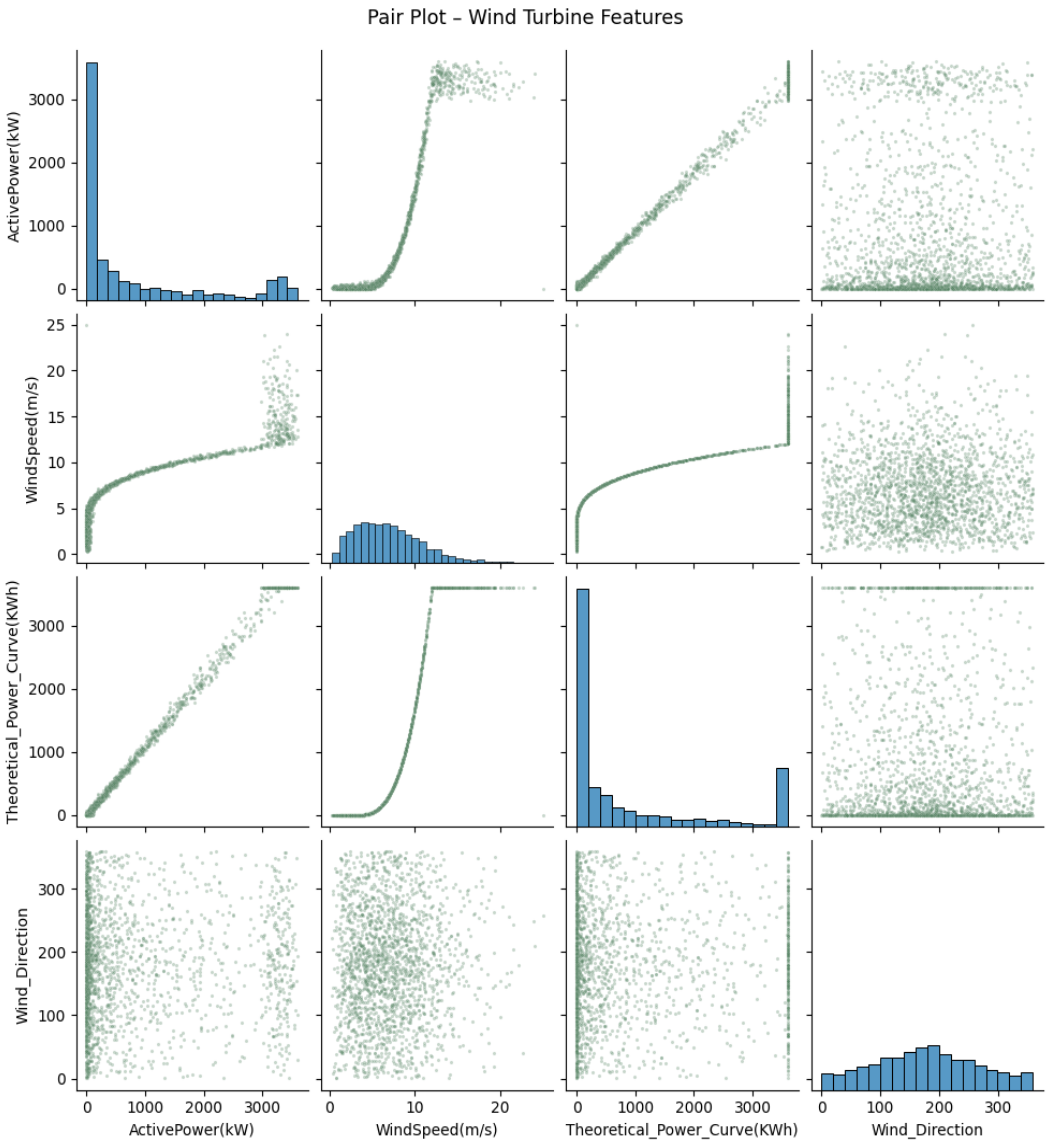

Figure 4: Comprehensive model evaluation plot.

8.2 User Interface Workflow

The AeroPredict web application was designed for intuitive usability. The following sequence demonstrates the end-to-end user workflow for generating a prediction.

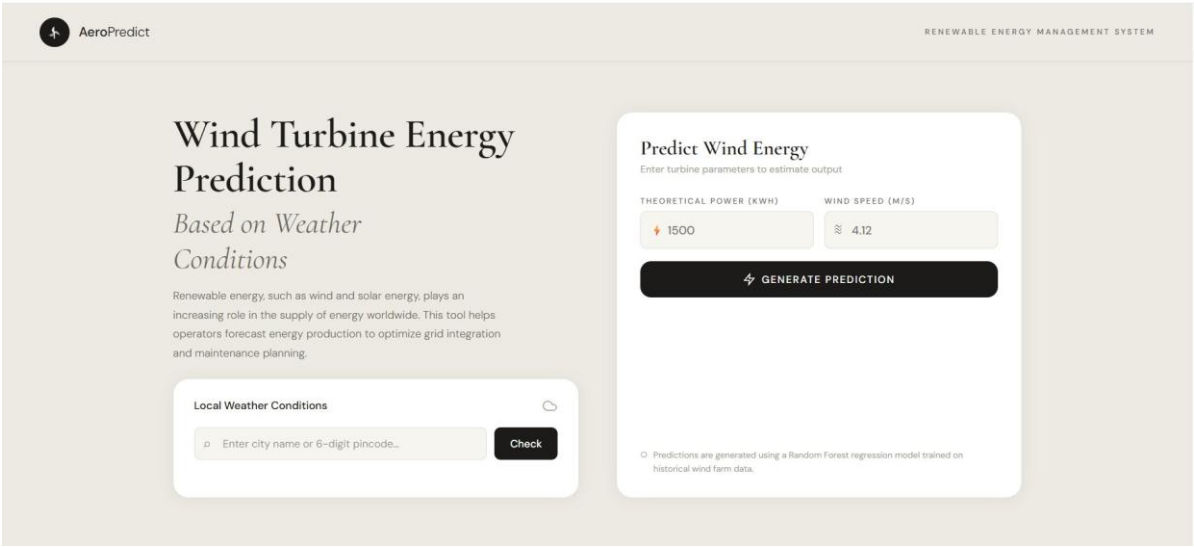

Figure 5: Initial Interface — clean dashboard with weather fetch module and turbine parameter input.

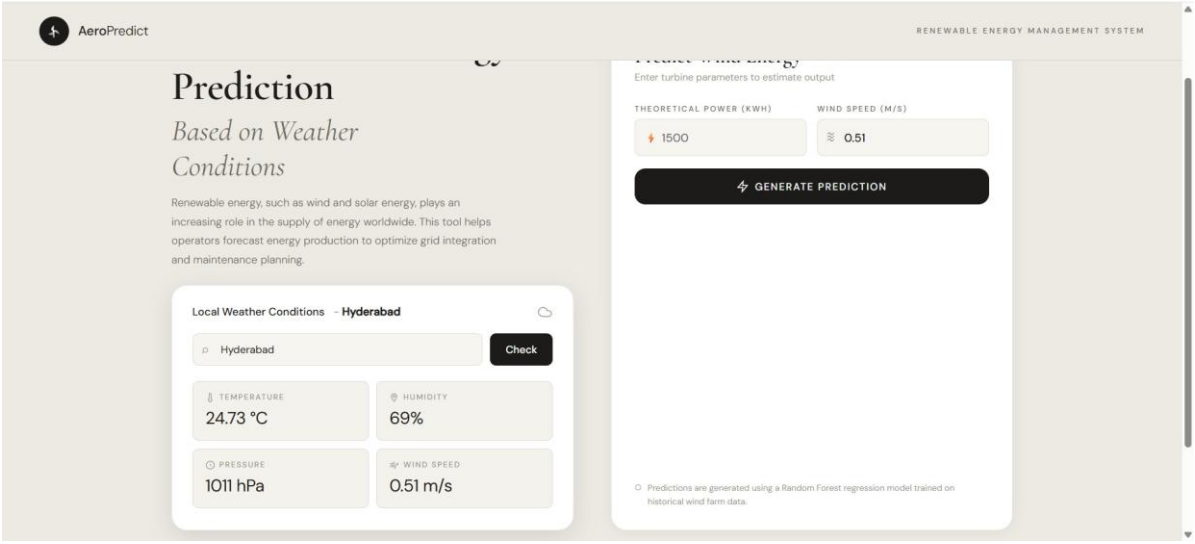

Figure 6: Real-Time Data Fetching — user inputs a location (e.g., Hyderabad) and live meteorological data is retrieved automatically.

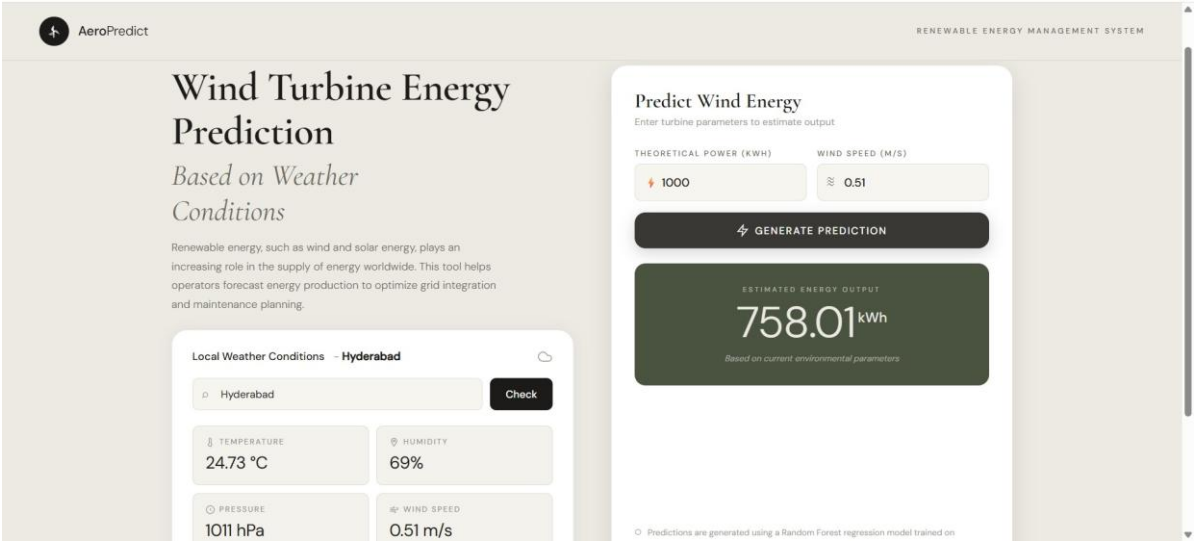

Figure 7: Model Inference & Output — the Flask backend processes inputs and returns estimated energy output (e.g., 758.01 kWh).

9. Future Work

- **Deep Learning Expansion:** Explore advanced time-series forecasting models such as Long Short-Term Memory (LSTM) networks to better capture the temporal dependencies in weather patterns.
- **Production Deployment:** Transition the Flask development server to production-grade WSGI servers (Gunicorn or uWSGI). Containerise the application using Docker and deploy to a cloud platform (AWS/GCP).
- **Multi-Turbine Scaling:** Expand the model to account for wake effects and aerodynamic interference across an entire array of wind turbines in a farm setting.

10. Conclusion

This project successfully demonstrates the technical and commercial feasibility of predicting wind turbine energy output using meteorological data. By establishing a robust machine learning pipeline and a highly usable Flask web interface, this solution offers a tangible method for improving energy grid management.

The integration of live API data bridges the gap between historical model training and real-time operational needs, providing a highly extensible framework for future research and development in the renewable energy sector.

11. References

- **GitHub Repository:** arrc10/Weather-Based-Prediction-of-Wind-Turbine-Energy-Output
- **Dataset:** Kaggle Wind Turbine SCADA Dataset — berkerisen/wind-turbine-scada-dataset
- **API:** OpenWeatherMap API — openweathermap.org